

# **“Media & Entertainment Mobile App for Subscriber Self-Service”**

**(StreamEase)**

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

## **BACHELOR OF TECHNOLOGY**

**In**

**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**

**By**

**SATWIKA GANDHE 2520090111**

**V. SIRI CHANDANA 2520090198**

Under the Esteemed Guidance of

**Divya Priya Degala**

**Assistant Professor**

**Department of Computer Science and Information Technology**



## **Koneru Lakshmaiah Education Foundation**

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, [www.klh.edu.in](http://www.klh.edu.in)

**K L (Deemed to be) University**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



**Declaration**

The Project Report entitled **“Media & Entertainment Mobile App for Subscriber Self-Service”** is a record of bonafide work of **SATWIKA GANDHE – 2520090111, V. SIRI CHANDANA – 2520090198** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team [Team No.], during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

**SATWIKA GANDHE – 2520090111**

**V. SIRI CHANDANA – 2520090198**

**K L (Deemed to be) University**  
**DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY**



This is to certify that the project report entitled **“Media & Entertainment Mobile App for Subscriber Self-Service”** is a bonafide work done and submitted by **SATWIK A GANDHE – 2520090111, V. SIRI CHANDANA – 2520090198** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team [Team No.], at KLH University during 2026–2027, Trimester 4.

**Signature of the Guide**

**Signature of the Course Coordinator**

**Signature of the HOD**

## **ACKNOWLEDGEMENT**

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another in the completion of this project.

Our greatest appreciation goes to our Course Coordinator, [Course Coordinator Name], and our guide, [Guide Name], Department of Computer Science and Information Technology, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to [HOD Name], Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

**SATWIKA GANDHE – 2520090111**

**V. SIRI CHANDANA – 2520090198**

## ABSTRACT

The Media & Entertainment Mobile App for Subscriber Self-Service (StreamEase) is a database-driven application designed to let entertainment subscribers manage their services independently. On existing media platforms, users often depend on customer support for routine tasks such as checking subscription plans, renewing services, viewing payment details, updating profiles and finding suitable content. When this information is scattered across screens, services and support channels, subscribers can easily miss payment deadlines, subscription expiry dates and relevant new content.

The proposed system provides a single platform where a subscriber can register and log in, browse and search Telugu movies, web series, TV shows and music, save titles to a watchlist, track viewing history, receive personalized recommendations based on genre, mood, rating and viewing history, manage subscription plans and transaction records, edit profile and preferences, and read notifications. A backend relational database is designed to store subscriber information, subscription details, payment transactions, content records, viewing history and user preferences securely, using relational tables, primary and foreign keys, normalization, SQL queries, transactions and integrity constraints.

The project was developed incrementally. A responsive frontend prototype built with HTML, CSS and JavaScript implements the complete subscriber flow — authentication screens, dashboard, browse and search with filters and sorting, watchlist and history, recommendations and a time-based content finder, subscription and profile management, and notifications — using in-memory demo data. In parallel, a normalized nine-table MySQL design with an ER model, schema, sample records, transaction logic and a REST API plan was prepared for the persistent implementation. The planned production architecture consists of an Android application, a Node.js and Express.js REST API, a MySQL database, and Firebase for authentication and push notifications. At the time of this report the frontend prototype is complete, and integration with the persistent backend is the next step. The system aims to reduce dependency on customer support, improve service efficiency, give subscribers easy access to their own information, and deliver a convenient and personalized entertainment experience.

# INDEX

| S.No. | Chapters                 | Topics                                                                                                                                                                   | Page No. |
|-------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|       | Acknowledgement          |                                                                                                                                                                          | 4        |
|       | Abstract                 |                                                                                                                                                                          | 5        |
| 1     | Introduction             | 1.1 Background of the Project<br>1.2 Problem Statement<br>1.3 Scope of the Project<br>1.4 Objectives<br>1.5 Importance of the Application<br>1.6 Target Users / Audience | 7        |
| 2     | System Requirements      | 2.1 Hardware Requirements<br>2.2 Software Requirements<br>2.3 Development Tools and Frameworks<br>2.4 Functional Requirements<br>2.5 Non-Functional Requirements         | 9        |
| 3     | Technology Stack         | 3.1 Frontend<br>3.2 Backend<br>3.3 Database<br>3.4 Version Control<br>3.5 Authentication and Security                                                                    | 11       |
| 4     | System Architecture      | 4.1 High-Level Architecture Diagram<br>4.2 Description of Each Layer<br>4.3 Deployment Architecture                                                                      | 12       |
| 5     | Design                   | 5.1 Database Design Principles<br>5.2 ER Diagram / Database Schema<br>5.3 Design Decisions and Assumptions                                                               | 14       |
| 6     | Implementation           | 6.1 Module-wise Implementation<br>6.2 Front-end Logic<br>6.3 API Integration<br>6.4 Authentication and Authorization<br>6.5 Database Logic: Transactions and Queries     | 17       |
| 7     | Features                 | 7.1 List of Main Features<br>7.2 How Each Feature Works                                                                                                                  | 19       |
| 8     | Testing                  | 8.1 Testing Approach<br>8.2 Integration and API Testing Plan<br>8.3 Tools Used for Testing<br>8.4 Test Cases and Results                                                 | 21       |
| 9     | Deployment               | 9.1 Steps to Deploy<br>9.2 Environment Configuration<br>9.3 Hosting of Frontend and Backend                                                                              | 23       |
| 10    | Challenges & Limitations | 10.1 Issues Faced During Development<br>10.2 Solutions Applied<br>10.3 Current Limitations or Known Bugs                                                                 | 24       |
| 11    | Future Enhancements      | 11.1 Planned Features<br>11.2 Possible Integrations or Optimizations                                                                                                     | 25       |
| 12    | Conclusion               | 12.1 Summary of the Project<br>12.2 What Was Achieved<br>12.3 Skills Learned                                                                                             | 26       |
| 13    | References               | Documentation, tools, and resources used                                                                                                                                 | 27       |
| 14    | Appendices               | A. Proposed SQL Schema and Sample Queries<br>B. Installation / Setup Instructions<br>C. Demo Login Details<br>D. API Reference Summary (Planned)<br>E. Screenshots       | 28       |

# CHAPTER 1: INTRODUCTION

## 1.1 Background of the Project

Subscription-based entertainment has become part of everyday life. Viewers watch movies, web series, television shows and music across several streaming services, and each service comes with its own account, subscription plan, billing cycle and viewing history. For a viewer who follows regional content such as Telugu cinema and series, titles are spread over platforms such as Netflix, Prime Video, aha and JioHotstar, which makes it hard to keep track of what has been watched, what has been saved for later and what to watch next.

In most platforms, routine tasks such as checking the current plan, finding the renewal date, reviewing past payments or correcting profile details still push the subscriber towards customer support. This is slow for the subscriber and costly for the service provider, because simple, repetitive requests occupy support staff who could be handling real problems. The Media & Entertainment Mobile App for Subscriber Self-Service, named **StreamEase**, was conceived to replace this dependence with a single self-service platform.

At the heart of the project is a relational database that acts as the single source of truth for every subscriber: profile, subscription, payments, saved titles, viewing history, preferences and notifications. The application layer reads this data to answer a subscriber's questions instantly and to generate personalized suggestions and timely alerts.

## 1.2 Problem Statement

Subscribers of media and entertainment services often depend on customer support for checking plans, renewing services, viewing payments, updating profiles and finding suitable content. Subscriber information, subscriptions, payments, viewing history and preferences become difficult to manage when the data is scattered. As a result, users may miss payment deadlines, subscription expiry dates or relevant new content.

Given a catalogue of entertainment content and a population of subscribers on different plans, the system must reliably answer three questions without human help: (i) what plan does this subscriber hold, when does it expire and what has been paid so far; (ii) what has this subscriber saved, watched and liked; and (iii) what should this subscriber watch next. Answering these questions manually does not scale, is error-prone, and gives subscribers no visibility into their own account.

## 1.3 Scope of the Project

The project covers the self-service lifecycle of a subscriber from the subscriber's own point of view: registering and logging in; browsing and searching content by title, genre and category; saving titles to a watchlist; tracking viewing history and continuing partly watched titles; receiving recommendations based on preferences, mood, ratings and history; viewing subscription plans, renewing or cancelling a plan and reviewing transaction history; editing profile and preferences; and reading notifications. The relational database design that supports these functions — tables, keys, constraints, transactions and queries — is part of the scope.

The current implementation (Project Review 2) is a responsive browser-based frontend prototype that uses in-memory demo data, while the persistent MySQL database, REST API

and Android client are designed and planned. Real payment processing, real video streaming, administration of the content catalogue and public cloud deployment are treated as out of scope for this submission.

## 1.4 Objectives

- **Discover:** browse Telugu movies, web series, TV shows and music from one place.
- **Search:** find content quickly by title, genre or content category, with filtering and sorting.
- **Personalize:** use genre, mood, ratings and viewing history to rank recommendations.
- **Self-service:** let subscribers manage profile, preferences, watchlist, history and subscription without contacting support.
- **Keep subscribers informed:** raise alerts for subscription expiry, payments and new content.
- **Store data reliably:** design a normalized relational schema with primary keys, foreign keys and constraints, and use transactions for multi-step operations such as renewal.
- **Responsive interface:** support desktop, tablet and mobile screen sizes.
- **Demonstrate:** show working frontend interactions suitable for project review, with a clear path to a persistent backend.

## 1.5 Importance of the Application

A self-service platform reduces dependence on customer support for routine tasks, improves service efficiency through instant access to account information, and gives subscribers control over their own entertainment experience. Personalized recommendations help subscribers find relevant content instead of searching across several services, and automatic alerts reduce accidental lapses caused by missed renewal dates. From an engineering point of view, the project also demonstrates how core database concepts — relational modelling, normalization, keys, SQL queries, transactions and integrity constraints — are applied to a realistic consumer application.

## 1.6 Target Users / Audience

The primary users are **subscribers**, who use the application to register, browse and search content, manage the watchlist and history, receive recommendations, manage the subscription and profile, and read notifications. The service provider's support team is an indirect beneficiary, since routine requests move into the self-service application. An administrative console for managing the content catalogue and plans is not part of the current scope.

## CHAPTER 2: SYSTEM REQUIREMENTS

### 2.1 Hardware Requirements

The application has modest hardware requirements. The Node.js backend and the MySQL database are lightweight relative to the dataset a subscriber platform generates in a course project, and the current frontend prototype runs entirely inside a web browser.

**Table 2.1: Minimum hardware requirements**

| Component      | Minimum requirement                                               |
|----------------|-------------------------------------------------------------------|
| Development PC | 8 GB or more RAM, Intel i5 (or equivalent) processor, SSD storage |
| Mobile device  | Android smartphone or emulator running Android 8 or later         |
| Network        | Stable internet connection for cloud database and API access      |

### 2.2 Software Requirements

- Android Studio — mobile application development platform
- Java or Kotlin — Android application programming; XML — interface layout design
- Node.js with npm and the Express.js framework — backend REST API
- MySQL 8.0 or later — relational database
- Firebase — user authentication and push notifications, where required
- Postman — API testing; Git and GitHub — version control and source management
- Visual Studio Code — backend and frontend development; Figma — UI/UX design
- A modern web browser (Chrome, Edge or Firefox) — to run the frontend prototype

### 2.3 Development Tools and Frameworks

Android Studio is the planned main platform for the mobile client, with Java or Kotlin for application logic and XML for screen layouts. Node.js with Express.js provides the API that connects the application to the database, and MySQL stores subscriber details, subscription plans, payments and entertainment content. Firebase can supply authentication and notifications. Postman is used to test backend APIs, Git and GitHub manage the source code, VS Code is the editor for backend development and Figma is used to design the user interface. The Review 2 frontend prototype itself was written in plain HTML, CSS and JavaScript.

### 2.4 Functional Requirements

**Table 2.2: Functional requirements**

| ID  | Requirement                                                                                                                                |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------|
| FR1 | A visitor can register with username, email and mobile number, log in with email and password, and request a forgot-password confirmation. |
| FR2 | A subscriber can browse content by category and genre, search by keyword, and sort the results.                                            |
| FR3 | A subscriber can add or remove titles from a watchlist.                                                                                    |

| ID  | Requirement                                                                                                                               |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------|
| FR4 | The system records viewing history with progress, shows Continue Watching, and lets the subscriber remove entries or clear the history.   |
| FR5 | The system recommends content using preferred genres, selected mood, ratings and viewing history, and offers a time-based content finder. |
| FR6 | A subscriber can view available plans and the active plan, renew or cancel the subscription, and see a transaction history.               |
| FR7 | A subscriber can edit profile details and preferences (genres, content type, language, notification settings).                            |
| FR8 | The system shows notifications and allows marking one or all as read, and clearing them.                                                  |

## 2.5 Non-Functional Requirements

- **Usability:** a consistent dashboard with sidebar navigation, top search, alerts and profile access.
- **Responsiveness:** layouts and navigation adapt to desktop, tablet and mobile screens.
- **Data integrity:** primary keys, foreign keys, UNIQUE and CHECK constraints keep stored data valid.
- **Security:** passwords are never stored in plain text, and protected data is available only to the authenticated owner.
- **Maintainability:** modular code organized by feature (authentication, home, browse, watchlist, subscription, profile, notifications).

## CHAPTER 3: TECHNOLOGY STACK

### 3.1 Frontend

The current frontend is a responsive single-page interface built with **HTML, CSS and JavaScript**. HTML defines the pages and sections, CSS provides a dark, professional theme with consistent cards, tables, modals and buttons together with media-query rules that adapt navigation and layouts for tablet and mobile screens, and JavaScript implements all interactions: search, filters, sorting, recommendations, watchlist, history, subscription actions and notifications. The prototype's design is organized as a sidebar navigation with a top search bar, alerts and profile access.

The user interface was designed in **Figma** before implementation. The planned production client is a native **Android** application written in Java or Kotlin, with XML layouts, which will reuse the same screens and flows validated in the prototype.

### 3.2 Backend

The planned backend is a REST API built with **Node.js and Express.js**. It will be organized into routes, controllers and models: routes declare the URL and required access, controllers validate requests and apply business rules, and models hold the SQL that reads and writes the database. This separation keeps each file focused on one responsibility. In the Review 2 prototype, the equivalent logic lives in JavaScript functions inside the frontend.

### 3.3 Database

**MySQL** is the relational database for the persistent system. The design contains nine related tables — users, content, plans, subscriptions, transactions, watchlist, watch\_history, preferences and notifications. Referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (for example one email per user and one watchlist entry per user and title) and data validity with CHECK constraints (for example rating and progress ranges, and an expiry date after the start date). In the current prototype, equivalent data is held in JavaScript arrays and objects, such as the `movies[]` array and a shared state object.

### 3.4 Version Control

**Git** is used for version control and **GitHub** for hosting the repository, so that both team members can work on frontend, backend and database scripts in parallel, review each other's changes and keep a history of the project.

### 3.5 Authentication and Security

In the prototype, login, registration and forgot-password flows are demonstration screens and no real credentials are verified. For the persistent system, **Firebase Authentication** is planned for sign-in and **Firebase Cloud Messaging** for push notifications, while the Express API will verify the identity of the caller on every protected route. Any password stored in MySQL will be stored only as a salted hash, never as plain text, and all SQL will use parameterized queries to prevent SQL injection.

## CHAPTER 4: SYSTEM ARCHITECTURE

### 4.1 High-Level Architecture Diagram

Two views of the architecture are given. Figure 4.1 shows the architecture of the frontend prototype implemented for Review 2, in which the data layer is held in JavaScript state. Figure 4.2 shows the planned production architecture, a three-tier design that separates the user interface, the business logic and the persisted data.

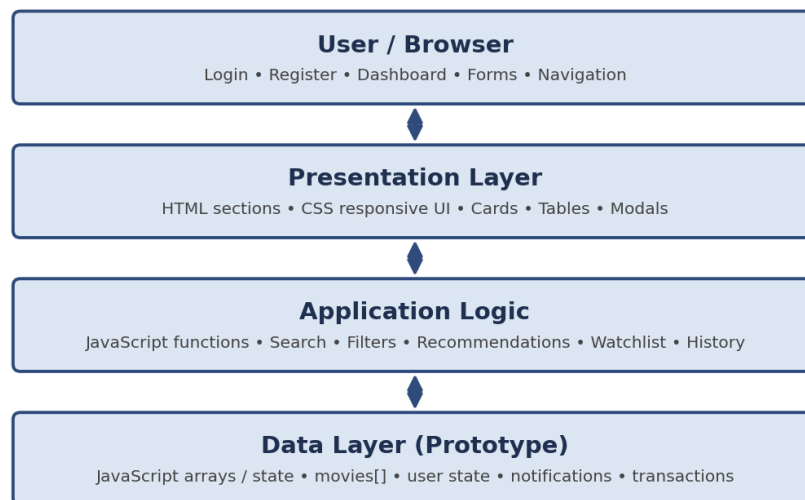


Figure 4.1: Layered architecture of the StreamEase frontend prototype

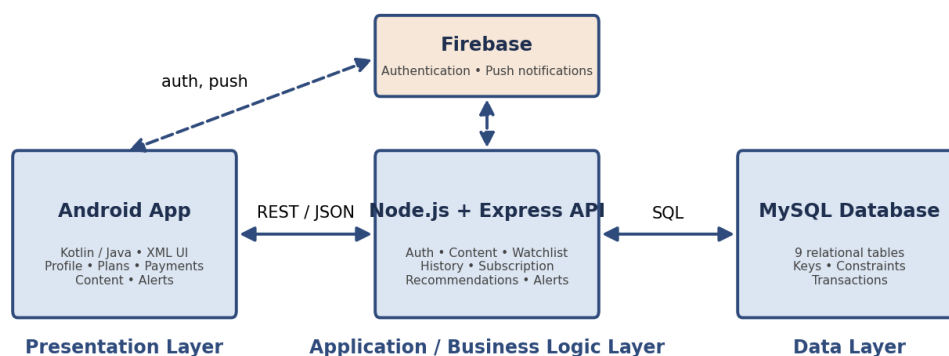


Figure 4.2: Planned three-tier architecture of the persistent system

### 4.2 Description of Each Layer

**Presentation layer.** In the prototype this layer consists of HTML pages and sections, responsive CSS, cards, tables and modals through which the subscriber logs in, registers, navigates the dashboard and fills in forms. In the planned system it is the Android application, which never talks to MySQL directly and obtains every piece of data from the REST API.

**Application logic layer.** In the prototype this is the set of JavaScript functions that implement search, filters, recommendations, watchlist and history. In the planned system it is the Express.js REST API: incoming requests pass through authentication middleware, then into a

controller that validates the request and applies business rules — for example, checking that a plan exists before creating a subscription — before reading or writing data through a model.

**Data layer.** In the prototype the data layer is made of JavaScript arrays and objects: `movies[]` for content metadata, plus a state object for the watchlist, history, mood, preferences, notifications and transactions. In the planned system it is the MySQL database, accessed through a connection pool so that multiple requests can be served concurrently.

**External services.** Firebase provides authentication and push notifications in the planned system. The prototype also provides external links to the OTT platforms Netflix, Prime Video, aha and JioHotstar, where titles are actually streamed.

### 4.3 Deployment Architecture

For development and demonstration, the current prototype runs as a single HTML file (`DBSE2.html`) opened in a web browser, with no server or database process. For the planned full system, the three tiers would run as separate local processes on a development machine — the Express server on a local port, MySQL on its default port 3306 and the Android application on an emulator or a connected phone. No containerization or CI/CD pipeline was configured, since the course scope did not call for cloud deployment; Chapter 11 outlines how the system could be extended to a hosted environment.

## CHAPTER 5: DESIGN

### 5.1 Database Design Principles

The schema is designed to be normalized so that each fact is stored in exactly one place and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. All attributes hold atomic values (first normal form). Attributes that describe a plan — its name, price, device limit and quality — live in the PLANS table instead of being repeated in every subscription, and attributes that describe a title — genre, rating, year and runtime — live in the CONTENT table instead of being repeated in every watchlist or history row, which removes partial and transitive dependencies (second and third normal forms).

Each table is also designed to enforce its own rules directly in the database using PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints, so that invalid data — a duplicate email, a rating outside the valid range, a progress value above 100 percent, or an expiry date before the start date — is rejected at the database level even if a bug elsewhere in the application tried to insert it.

### 5.2 ER Diagram / Database Schema

The conceptual database consists of nine related tables, shown in Figure 5.1. The Review 2 ER model presented seven core entities; the notification and transaction entities, which appear in the Review 2 schema, are included here to give the complete design.

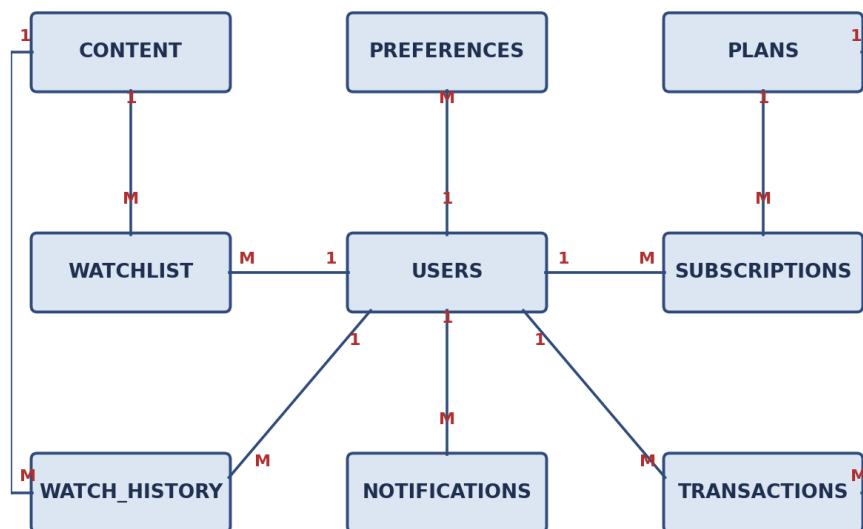


Figure 5.1: Entity-Relationship diagram of the 9-table database schema

USERS stores every subscriber account. CONTENT holds the entertainment catalogue. PLANS holds the subscription plans on offer. SUBSCRIPTIONS links a user to a plan for a period of time with a status. TRANSACTIONS is the ledger of payments made. WATCHLIST and WATCH\_HISTORY are junction tables that connect users to the content they saved or

watched, and the history stores progress. PREFERENCES stores a user's preferred genres, content types and languages, and NOTIFICATIONS stores alerts with a read status.

**Table 5.1: Database tables and key fields**

| Table         | Key fields                                                                        |
|---------------|-----------------------------------------------------------------------------------|
| USERS         | user_id (PK), name, email, mobile, password (stored as a hash)                    |
| CONTENT       | content_id (PK), title, genre, category, rating, year, runtime                    |
| PLANS         | plan_id (PK), plan_name, price, device_limit, quality                             |
| SUBSCRIPTIONS | subscription_id (PK), user_id (FK), plan_id (FK), start_date, expiry_date, status |
| TRANSACTIONS  | transaction_id (PK), user_id (FK), plan_id (FK), date, amount, status             |
| WATCHLIST     | watchlist_id (PK), user_id (FK), content_id (FK)                                  |
| WATCH_HISTORY | history_id (PK), user_id (FK), content_id (FK), progress, watched_date            |
| PREFERENCES   | preference_id (PK), user_id (FK), genre, content_type, language                   |
| NOTIFICATIONS | notification_id (PK), user_id (FK), type, message, read_status                    |

**Table 5.2: Sample database records (from the Review 2 design)**

| Table         | ID | User / Content ID  | Key value                 | Status / Detail             |
|---------------|----|--------------------|---------------------------|-----------------------------|
| USERS         | 1  | –                  | user@streamease.com       | +91 90000 00000             |
| CONTENT       | 1  | –                  | RRR                       | Action, Drama • rating 4.8  |
| CONTENT       | 8  | –                  | Sita Ramam                | Romance, Drama • rating 4.7 |
| SUBSCRIPTIONS | 1  | User 1             | Premium                   | Active                      |
| WATCHLIST     | 1  | User 1 / Content 8 | Saved title               | Saved                       |
| WATCH_HISTORY | 1  | User 1 / Content 9 | Progress                  | 72%                         |
| PREFERENCES   | 1  | User 1             | Action, Thriller, Romance | Language: Telugu            |
| NOTIFICATIONS | 1  | User 1             | Recommendation            | Unread                      |

### 5.3 Design Decisions and Assumptions

- **SUBSCRIPTIONS vs. TRANSACTIONS:** a subscription records what the subscriber is entitled to and until when, while a transaction records a payment event. They are kept separate so that one subscription can be backed by many payments over time, and both are written together inside one database transaction at renewal so they can never disagree.
- **Junction tables for watchlist and history:** the relationship between users and content is many-to-many, so it is expressed through WATCHLIST and WATCH\_HISTORY rather than by repeating content details. WATCHLIST carries a UNIQUE constraint on (user\_id, content\_id), so a title cannot be saved twice.

- **Table name USERS:** the Review 2 entity USER is named USERS in the SQL schema to avoid clashing with a MySQL keyword.
- **Password handling:** the Review 2 schema lists a password field; in the persistent design this column stores only a hash.
- **Derived data is not stored:** values such as total watch time, completed titles and days to expiry are calculated from history and subscription rows when needed rather than stored.
- **Prototype data:** in Review 2 the data layer is JavaScript state, and plan and transaction records are generated by frontend functions. It is presented as the current prototype data design, not as a completed backend database.

## CHAPTER 6: IMPLEMENTATION

### 6.1 Module-wise Implementation

The frontend prototype is organized into eight functional modules, shown in Table 6.1. Each module is a section of the single-page interface with its own JavaScript functions that read and update the shared state.

**Table 6.1: Modules of the StreamEase prototype**

| Module                | Functionality                                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication        | Email and password login, registration with username, email and mobile number, and a forgot-password confirmation demo.                     |
| Home                  | Welcome dashboard with quick stats (watch time, completed titles), Continue Watching, Recommended for You, and a time-based content finder. |
| Browse & Search       | Category and genre filters, sorting, and keyword search over movies, web series, TV shows and music.                                        |
| Watchlist & History   | Save titles, resume from Continue Watching, remove an entry, and clear the history.                                                         |
| Recommendations       | Ranking of content by preferences, selected mood, ratings and viewing history.                                                              |
| Subscription          | Plans, the active plan, renewal and cancellation demo, and transaction history.                                                             |
| Profile & Preferences | Edit profile, password change demo, preferred genres, language and notification settings.                                                   |
| Notifications         | Alerts list with mark as read, mark all as read and clear notifications.                                                                    |

The most involved piece of logic is **recommendation ranking**. Rather than relying on a single signal, the ranking combines several: whether a title's genre matches the subscriber's saved preferences, the mood the subscriber selected on the dashboard, the title's rating, and what the subscriber has already watched, so that finished titles are not promoted again and titles similar to watched ones are favoured. The **time-based content finder** takes the time a subscriber has available and suggests titles whose runtime fits within it.

### 6.2 Front-end Logic

The frontend keeps its data in two places: the `movies[]` array, which holds content metadata (title, genre, category, rating, year, runtime and description), and a shared state object, which holds the watchlist IDs, viewing history, selected mood, preferences and notifications. Every module reads from and writes to this shared state, and the interface is re-rendered after each change, so that, for example, adding a title to the watchlist immediately updates the watchlist page and the quick stats. Subscription plans and transaction records are produced by frontend functions.

The interface itself is built from a small set of reusable components — cards, tables, buttons and modals — styled once in CSS so that every page loads, confirms actions and displays

results consistently. Responsive CSS rules change the navigation and the layouts for smaller screens, replacing the sidebar with a mobile navigation on narrow devices.

### 6.3 API Integration

The Review 2 prototype does not call a backend, and its data actions run entirely in the browser. For the persistent system, each frontend function that currently changes JavaScript state will be replaced by a call to a REST endpoint, and the Express API will return JSON. A summary of the planned endpoints is given in Appendix D. The planned convention is that every protected request carries the signed-in user's token, a missing or expired token returns 401 Unauthorized, and invalid input returns 400 Bad Request.

### 6.4 Authentication and Authorization

In the prototype, authentication is a user-interface demonstration: the login and registration screens validate the form and move the user into the dashboard, but no credentials are checked against stored accounts. In the persistent system, a subscriber will register and sign in through Firebase Authentication, the API will verify the token on every protected route, and every query will be scoped by the authenticated user's ID, so a subscriber can read and change only their own profile, subscription, history, preferences and notifications.

### 6.5 Database Logic: Transactions and Queries

The persistent design uses SQL transactions wherever several tables must change together. **Subscription renewal** is the main example: recording the payment, extending the subscription and creating the confirmation notification must either all succeed or all be rolled back, so that a subscriber is never charged without a renewed plan, or given a renewed plan without a payment record. The complete transaction, the expiry-alert query and the recommendation query are listed in Appendix A.

Other queries planned for the system include searching content by title, genre or category with sorting by rating; listing a subscriber's watchlist and history by joining them to CONTENT; calculating total watch time and completed titles from history; finding subscriptions that expire within a few days so that expiry notifications can be created; and returning a subscriber's transaction history in date order.

## CHAPTER 7: FEATURES

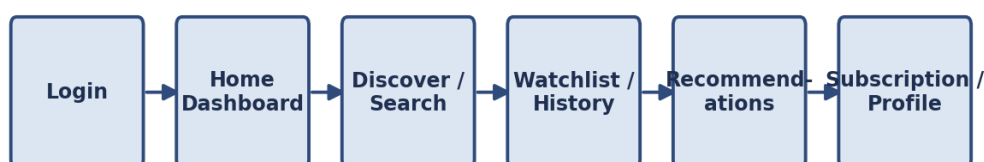
### 7.1 List of Main Features

Subscriber-facing features implemented in the frontend prototype:

- Login, registration (username, email, mobile) and forgot-password confirmation
- Home dashboard with quick stats, Continue Watching and Recommended for You
- Browse Telugu movies, web series, TV shows and music, with category and genre filters, sorting and keyword search
- Watchlist: save and remove titles
- Viewing history: continue watching, remove an entry, clear history
- Personalized recommendations from preferences, mood, ratings and history, and a time-based content finder
- Subscription management: plans, active plan, renewal and cancellation demo, transaction history
- Profile editing, password change demo and preferences (genres, language, notifications)
- Notifications: mark as read, mark all as read and clear
- External links to Netflix, Prime Video, aha and JioHotstar
- Responsive layout for desktop, tablet and mobile

Planned for the persistent system: secure authentication, storage of all data in MySQL, automatic expiry and payment alerts generated from the database, and push notifications for new content.

### 7.2 How Each Feature Works



*Figure 7.1: Core user flow of StreamEase*

**Authentication.** A visitor signs in with email and password or registers with username, email and mobile number. A forgot-password request shows a confirmation message. After a successful login the user reaches the home dashboard.

**Home dashboard.** The dashboard welcomes the subscriber and summarizes viewing through quick stats such as watch time and completed titles. It lists Continue Watching items and a Recommended for You row, and includes the time-based finder: the subscriber enters the time available and the system suggests titles that fit.

**Browse and search.** The subscriber selects a category or genre, types a keyword, or sorts the list, and the visible titles update instantly. Each title shows its genre, rating, year and runtime

and can be added to the watchlist; external OTT links let the subscriber open the title on the streaming platform that carries it.

**Watchlist and history.** Saving a title adds its ID to the subscriber's watchlist, and removing it deletes the entry. Viewing history stores what was watched and how far, so the dashboard can offer Continue Watching; the subscriber can remove a single entry or clear the entire history.

**Recommendations.** Titles are scored using the subscriber's preferred genres, the selected mood, the rating of each title and the existing history, and the highest-ranked titles are shown. Changing the preferences or the mood changes the ranking immediately.

**Subscription.** The subscription page shows the available plans and the active plan. A renewal or cancellation updates the plan status in the demo and adds an entry to the transaction history, which lists the date, plan, amount and status of each payment.

**Profile, preferences and notifications.** The profile page edits personal details and preferences. Notifications appear in the alerts area; a subscriber can mark one alert or all alerts as read, or clear them.

## CHAPTER 8: TESTING

### 8.1 Testing Approach

The Review 2 deliverable is a frontend prototype, so testing so far is **manual functional testing in the browser**: each module is exercised through its screens, and the result of every interaction is compared with the expected behaviour. Responsive behaviour is checked by resizing the browser window to desktop, tablet and mobile widths. Because the prototype keeps its data in memory, the tests also confirm that state changes in one module are reflected in the others, for example that a title added to the watchlist appears on the watchlist page and in the quick stats.

### 8.2 Integration and API Testing Plan

When the persistent backend is connected, testing will follow the same incremental approach: each backend module will be verified against a real running MySQL database before the next module is built on it, rather than against mocked data. A Postman collection will cover every REST endpoint, organized by resource, with the login request saving the returned token into a collection variable so that requests can be chained into a complete workflow without copying values by hand. As a final integration check, the database will be dropped and rebuilt from the schema and seed scripts, and the full checklist re-run on the fresh installation.

### 8.3 Tools Used for Testing

- Web browser and developer tools — functional and responsive testing of the frontend prototype
- Postman — manual and scripted API testing (backend phase)
- MySQL command-line client or MySQL Workbench — verification of table contents and constraint behaviour (backend phase)
- GitHub — tracking and re-testing of defects

### 8.4 Test Cases and Results

Table 8.1 lists the test cases for the implemented frontend prototype. The Actual Result column is to be completed by the team as each case is executed on the final build. Table 8.2 lists the database and API test cases planned for the backend phase.

**Table 8.1: Frontend prototype test cases**

| # | Test case                                           | Expected result                                           | Actual result |
|---|-----------------------------------------------------|-----------------------------------------------------------|---------------|
| 1 | Log in with a valid email and password              | Dashboard opens                                           |               |
| 2 | Submit the login form with empty fields             | Login is not allowed; the user stays on the login screen  |               |
| 3 | Register with username, email and mobile            | Registration completes and the user can proceed to log in |               |
| 4 | Use Forgot password                                 | A confirmation message is shown                           |               |
| 5 | Search for a known title keyword (for example, RRR) | Only matching titles are listed                           |               |

| #  | Test case                                                  | Expected result                                                           | Actual result |
|----|------------------------------------------------------------|---------------------------------------------------------------------------|---------------|
| 6  | Filter by genre or category and sort by rating             | The list updates to the filter and the sort order                         |               |
| 7  | Add a title to the watchlist, then remove it               | Title appears in the watchlist, then disappears                           |               |
| 8  | Open a title and return to the dashboard                   | History is updated and Continue Watching shows the title                  |               |
| 9  | Clear viewing history                                      | History list is empty                                                     |               |
| 10 | Select a mood or enter available time in the finder        | Recommendations / suggestions change accordingly                          |               |
| 11 | Edit profile details and preferences                       | Changes are shown on the profile page; recommendations use the new genres |               |
| 12 | Renew or cancel the subscription (demo)                    | Plan status changes and a transaction entry is added                      |               |
| 13 | Mark one notification read, then mark all read, then clear | Read status and counts update; list is cleared                            |               |
| 14 | Resize the window to a mobile width                        | Navigation changes to the mobile layout; no content is cut off            |               |
| 15 | Refresh the page after making changes                      | Demo state resets (known prototype limitation)                            |               |

**Table 8.2: Planned database and API test cases (backend phase)**

| #  | Test case                                                  | Expected result                                          | Status  |
|----|------------------------------------------------------------|----------------------------------------------------------|---------|
| 1  | Register with an email that already exists                 | Rejected by the UNIQUE constraint with a clear message   | Planned |
| 2  | Insert a subscription for a non-existent plan              | Rejected by the foreign key                              | Planned |
| 3  | Add the same title to the watchlist twice                  | Second insert rejected (UNIQUE user_id + content_id)     | Planned |
| 4  | Save watch progress above 100                              | Rejected by the CHECK constraint                         | Planned |
| 5  | Create a subscription whose expiry is before its start     | Rejected by the CHECK constraint                         | Planned |
| 6  | Renewal fails after the payment row is inserted            | Whole transaction rolls back; no partial rows remain     | Planned |
| 7  | Run the expiry query for a subscription ending in two days | The subscription is returned and an alert can be created | Planned |
| 8  | Call a protected endpoint without a token                  | 401 Unauthorized                                         | Planned |
| 9  | Request another user's watchlist                           | Only the caller's own rows are returned                  | Planned |
| 10 | Delete a user who has history and notifications            | Dependent rows are removed; zero orphaned rows           | Planned |

# CHAPTER 9: DEPLOYMENT

## 9.1 Steps to Deploy

**Running the current prototype** requires no installation:

- Place the prototype file (DBSE2.html) on the computer.
- Double-click the file, or open it with Chrome, Edge or Firefox.
- Use the demo account to log in, or register a new demo account, and explore the dashboard.

**Deploying the planned full system** on a development machine:

- Load the schema and seed data into MySQL: `mysql -u root -p < database/schema.sql`, then `... < database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev`
- Build and run the Android application from Android Studio on an emulator or a connected phone running Android 8 or later
- Configure Firebase for authentication and notifications

## 9.2 Environment Configuration

The prototype needs no configuration. For the planned backend, all server configuration will be kept in a single `.env` file that is never committed to version control, covering the database connection (host, port, user, password and database name), the port on which the server listens, and the Firebase credentials. A `.env.example` template with placeholder values will be committed instead, so that the required variables are documented without exposing real credentials.

## 9.3 Hosting of Frontend and Backend

For this submission, the frontend prototype runs locally in a browser, which is appropriate for development and in-class demonstration. The backend and database are planned to run locally as a Node.js process and a local MySQL instance. A production deployment was not required for this course submission; Chapter 11 outlines how the same codebase could be adapted to a hosted environment.

## CHAPTER 10: CHALLENGES & LIMITATIONS

### 10.1 Issues Faced During Development

- Keeping navigation and multiple dashboard modules consistent inside a single-page frontend.
- Managing shared frontend state for the watchlist, history, preferences and notifications, so that a change made in one module is visible in the others.
- Designing a recommendation ranking from multiple signals such as rating, genre, mood and viewing history.
- Making the interface responsive for desktop, tablet and mobile screen sizes.
- Reconciling the scope across reviews: the abstract describes a mobile app backed by a database, while the Review 2 deliverable is a browser-based frontend prototype with demo data.

### 10.2 Solutions Applied

To keep navigation consistent, every module is built as a section of one single-page interface with a shared sidebar, top search bar and alerts area. To manage shared state, the watchlist, history, mood, preferences and notifications are kept in one state object that all modules read and write, with the interface re-rendered after each change. For recommendations, several signals are combined into one ranking instead of depending on a single attribute. For responsiveness, CSS media-query rules change the navigation and layouts at smaller widths. To reconcile the scope, the report states clearly which parts are implemented (the prototype) and which are designed and planned (the database, API and Android client), and the ER model, schema and sample records were prepared so that the next phase can start directly from them.

### 10.3 Current Limitations or Known Bugs

- Data and account actions are demo state held in the browser; nothing is persisted, and a page refresh resets the data.
- There is no secure authentication or password reset; login and registration are demonstration screens.
- The persistent backend database is not yet connected to the prototype.
- There is no real payment gateway and no real streaming; subscription actions are demonstrations, and titles are opened on external OTT platforms.
- The current deliverable is a browser-based interface rather than a native Android application.
- Recommendations are based on a small sample catalogue and have not been tuned with real usage data.

# CHAPTER 11: FUTURE ENHANCEMENTS

## 11.1 Planned Features

- Connect the frontend to a persistent MySQL database through the Express.js REST API.
- Implement secure authentication and password reset, with hashed passwords and token-based sessions.
- Store users, preferences, watchlist and history persistently, so that data survives page reloads and devices.
- Add real subscription and payment integration through a payment gateway.
- Integrate real content and catalogue APIs in place of the sample catalogue.
- Improve the recommendation model using real usage data.
- Automatic notifications for subscription expiry, payments and new content, delivered as push notifications.

## 11.2 Possible Integrations or Optimizations

- Build the native Android application with Kotlin and Jetpack-style components or XML layouts, reusing the screens validated in the prototype.
- Use Firebase Authentication and Firebase Cloud Messaging for sign-in and push alerts.
- Add indexes on frequently searched columns such as content title and genre, and on subscription expiry date.
- Cache frequently requested catalogue queries to reduce repeated database load.
- Containerize the backend and database with Docker for a consistent, reproducible deployment.
- Add an automated test suite (for example, Jest and Supertest) to complement manual and Postman testing.

## **CHAPTER 12: CONCLUSION**

### **12.1 Summary of the Project**

This project set out to replace scattered, support-dependent subscriber management with a single database-driven self-service application for media and entertainment. A responsive frontend prototype, StreamEase, was designed and built to demonstrate the full subscriber flow, and a normalized nine-table relational design with an ER model, schema, sample records, transaction logic and API plan was prepared for the persistent implementation.

### **12.2 What Was Achieved**

The prototype delivers authentication screens, a dashboard, browse, search, filters and sorting, a watchlist and viewing history, recommendations with a time-based finder, subscription, profile and preference management, and notifications, all in a consistent and responsive interface. The database side delivers a normalized design with primary and foreign keys, integrity constraints, a defined transaction for subscription renewal, and sample queries for expiry alerts and recommendations. The remaining step is to connect the interface to the persistent backend.

### **12.3 Skills Learned**

Working on this project gave practical experience in requirement analysis for a consumer application; relational database design, ER modelling and normalization; defining constraints at the database layer rather than trusting application code alone; planning SQL transactions for multi-table operations; designing a REST API; building a responsive single-page interface with shared state in JavaScript; designing screens in Figma; using Git and GitHub for teamwork; and planning a systematic testing process.

## CHAPTER 13: REFERENCES

- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- Android Developers — Documentation, <https://developer.android.com/docs>
- Firebase — Documentation (Authentication and Cloud Messaging), <https://firebase.google.com/docs>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- Postman Learning Center, <https://learning.postman.com>
- Figma — Help Center, <https://help.figma.com>
- GitHub Docs, <https://docs.github.com>
- MDN Web Docs — HTML, CSS and JavaScript reference, <https://developer.mozilla.org>

## CHAPTER 14: APPENDICES

### Appendix A: Proposed SQL Schema and Sample Queries

The proposed MySQL schema for the persistent implementation:

```
CREATE TABLE users (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  mobile VARCHAR(15) UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE content (
  content_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(150) NOT NULL,
  genre VARCHAR(50) NOT NULL,
  category ENUM('Movie','Web Series','TV Show','Music') NOT NULL,
  rating DECIMAL(2,1) CHECK (rating BETWEEN 0 AND 5),
  release_year YEAR, runtime_min INT CHECK (runtime_min > 0),
  description TEXT
);
CREATE TABLE plans (
  plan_id INT AUTO_INCREMENT PRIMARY KEY,
  plan_name VARCHAR(50) NOT NULL UNIQUE,
  price DECIMAL(8,2) NOT NULL CHECK (price >= 0),
  device_limit INT NOT NULL, quality VARCHAR(20)
);
CREATE TABLE subscriptions (
  subscription_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL, plan_id INT NOT NULL,
  start_date DATE NOT NULL, expiry_date DATE NOT NULL,
  status ENUM('Active','Expired','Cancelled') NOT NULL,
  CHECK (expiry_date > start_date),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (plan_id) REFERENCES plans(plan_id)
);
CREATE TABLE transactions (
  transaction_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL, plan_id INT NOT NULL,
  txn_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  amount DECIMAL(8,2) NOT NULL,
  status ENUM('Success','Failed','Pending') NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (plan_id) REFERENCES plans(plan_id)
);
CREATE TABLE watchlist (
  watchlist_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL, content_id INT NOT NULL,
  UNIQUE (user_id, content_id),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (content_id) REFERENCES content(content_id) ON DELETE CASCADE
);
CREATE TABLE watch_history (
  history_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL, content_id INT NOT NULL,
  progress TINYINT NOT NULL CHECK (progress BETWEEN 0 AND 100),
  watched_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
  FOREIGN KEY (content_id) REFERENCES content(content_id) ON DELETE CASCADE
);
CREATE TABLE preferences (
  preference_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL, genre VARCHAR(50),
  content_type VARCHAR(30), language VARCHAR(30),
  FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
CREATE TABLE notifications (
  notification_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  type ENUM('Expiry','Payment','New Content','Recommendation') NOT NULL,
```

```
message VARCHAR(255) NOT NULL,
read_status BOOLEAN NOT NULL DEFAULT FALSE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

Subscription renewal as one atomic transaction (the ? marks are bound parameters):

```
START TRANSACTION;
INSERT INTO transactions (user_id, plan_id, amount, status)
  SELECT ?, plan_id, price, 'Success' FROM plans WHERE plan_id = ?;
UPDATE subscriptions
  SET plan_id = ?, start_date = CURDATE(),
    expiry_date = DATE_ADD(CURDATE(), INTERVAL 1 MONTH), status = 'Active'
  WHERE user_id = ?;
INSERT INTO notifications (user_id, type, message)
  VALUES (?, 'Payment', 'Your subscription was renewed successfully.');
```

COMMIT; -- ROLLBACK if any statement fails

Subscriptions that expire within three days (used to create expiry alerts):

```
SELECT s.user_id, s.expiry_date
FROM subscriptions s
WHERE s.status = 'Active'
  AND s.expiry_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 3 DAY);
```

Recommendations from preferred genres, excluding already watched titles:

```
SELECT c.content_id, c.title, c.genre, c.rating
FROM content c JOIN preferences p ON c.genre = p.genre
WHERE p.user_id = ?
  AND c.content_id NOT IN (SELECT content_id FROM watch_history WHERE user_id = ?)
ORDER BY c.rating DESC LIMIT 10;
```

## Appendix B: Installation and Setup Instructions

Frontend prototype:

- Copy DBSE2.html to any folder.
- Open it in Chrome, Edge or Firefox.

Planned full system:

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev (backend)`
- Open the Android project in Android Studio and run it on an emulator or device

## Appendix C: Demo Login Details

The Review 2 sample records contain the following demo account. Enter the password that is configured in your prototype build.

| Role              | Email               | Password                         |
|-------------------|---------------------|----------------------------------|
| Subscriber (demo) | user@streamease.com | [as configured in the prototype] |

## Appendix D: API Reference Summary (Planned)

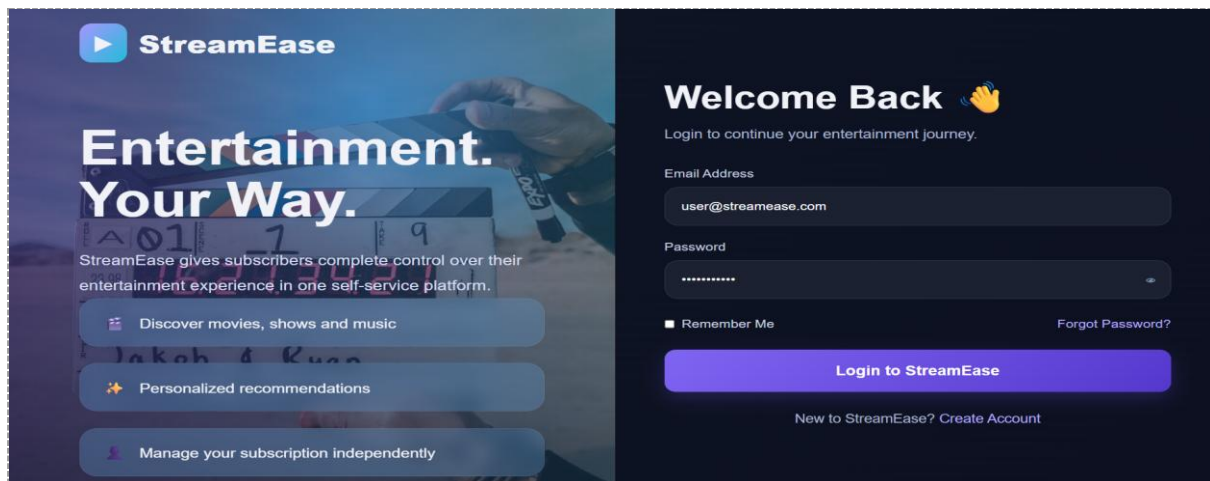
The following REST endpoints are planned for the persistent system. They are not part of the Review 2 prototype.

| Resource              | Endpoints                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| Auth                  | POST /api/auth/register, POST /api/auth/login, GET /api/auth/me                                                     |
| Content               | GET /api/content (search, filter, sort), GET /api/content/:id                                                       |
| Watchlist             | GET /api/watchlist, POST /api/watchlist, DELETE /api/watchlist/:contentId                                           |
| History               | GET /api/history, POST /api/history, DELETE /api/history/:id, DELETE /api/history                                   |
| Recommendations       | GET /api/recommendations                                                                                            |
| Plans & Subscription  | GET /api/plans, GET /api/subscription, POST /api/subscription/renew, PUT /api/subscription/cancel                   |
| Transactions          | GET /api/transactions                                                                                               |
| Profile & Preferences | GET /api/profile, PUT /api/profile, GET /api/preferences, PUT /api/preferences                                      |
| Notifications         | GET /api/notifications, PUT /api/notifications/:id/read, PUT /api/notifications/read-all, DELETE /api/notifications |

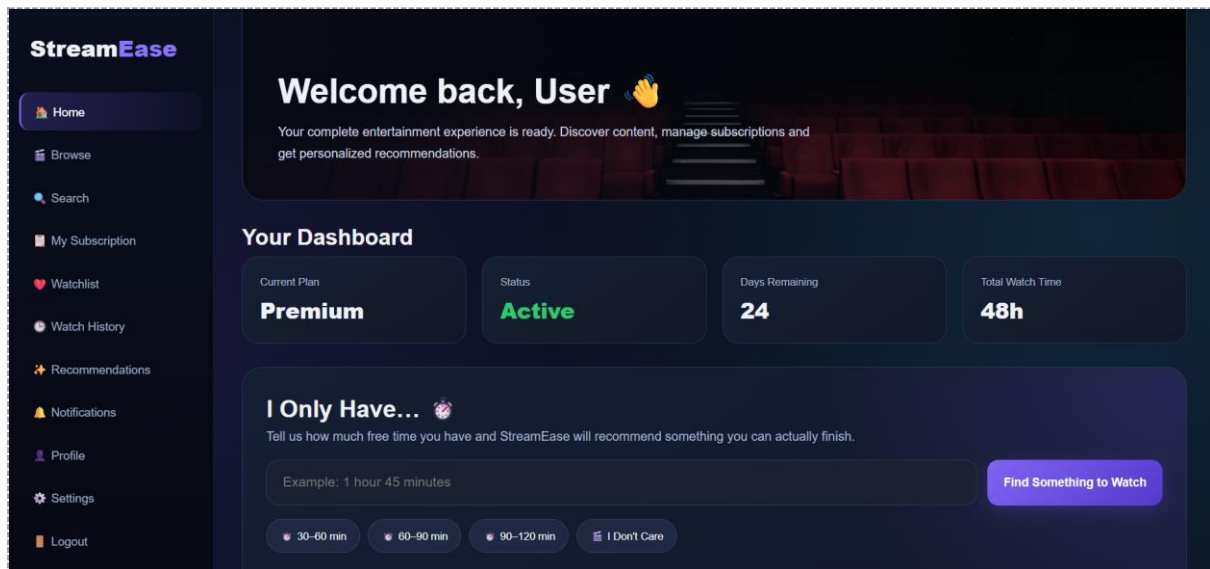
## Appendix E: Screenshots

Insert actual screenshots from the running prototype (DBSE2.html) before submission. Do not use placeholder images in the final document. Suggested screenshots, in order:

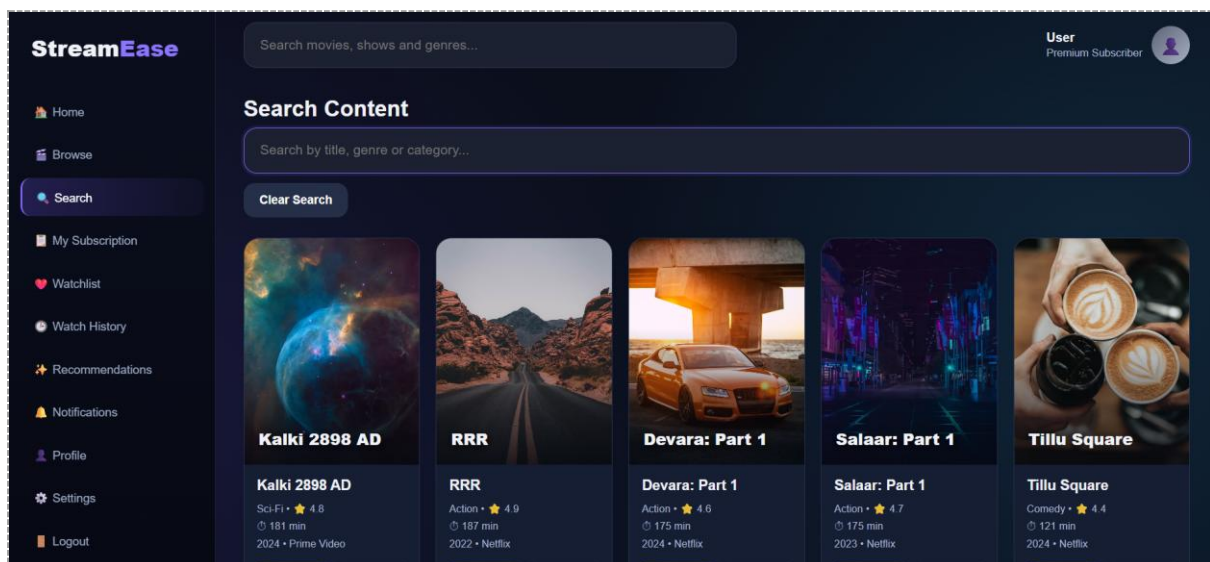
### 1. Login / Registration screen



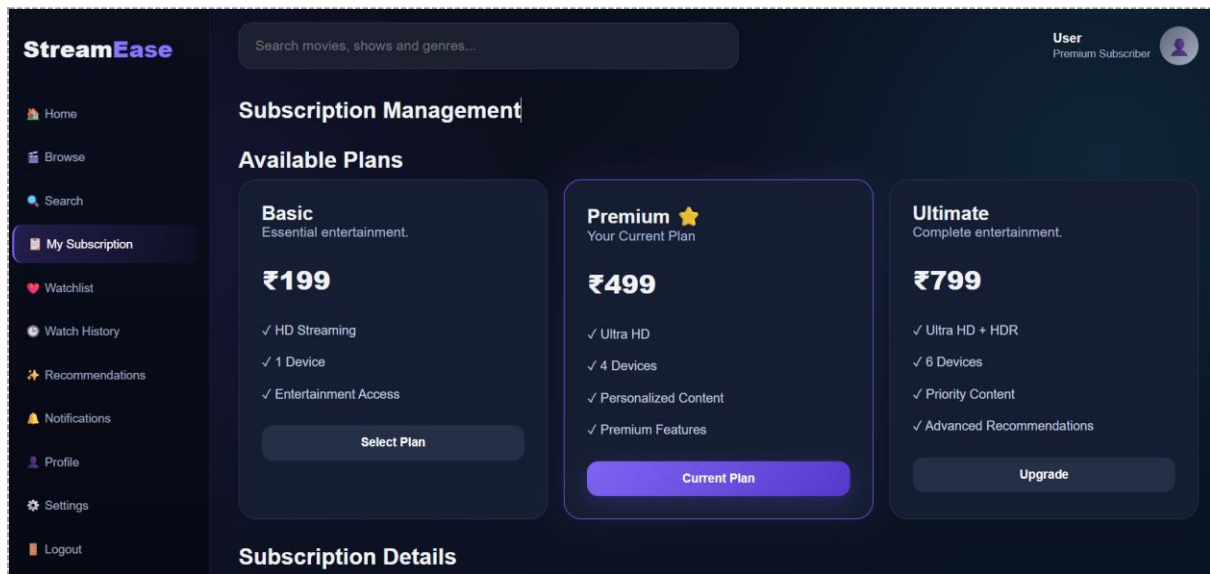
## 2. Home dashboard



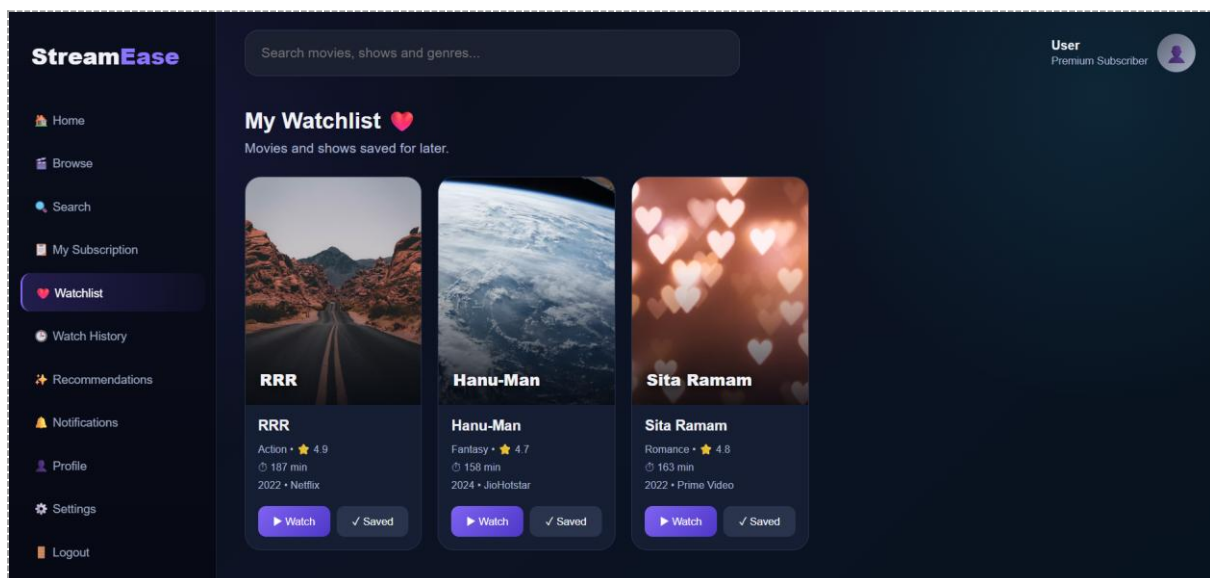
## 3. Browse / Search with filters



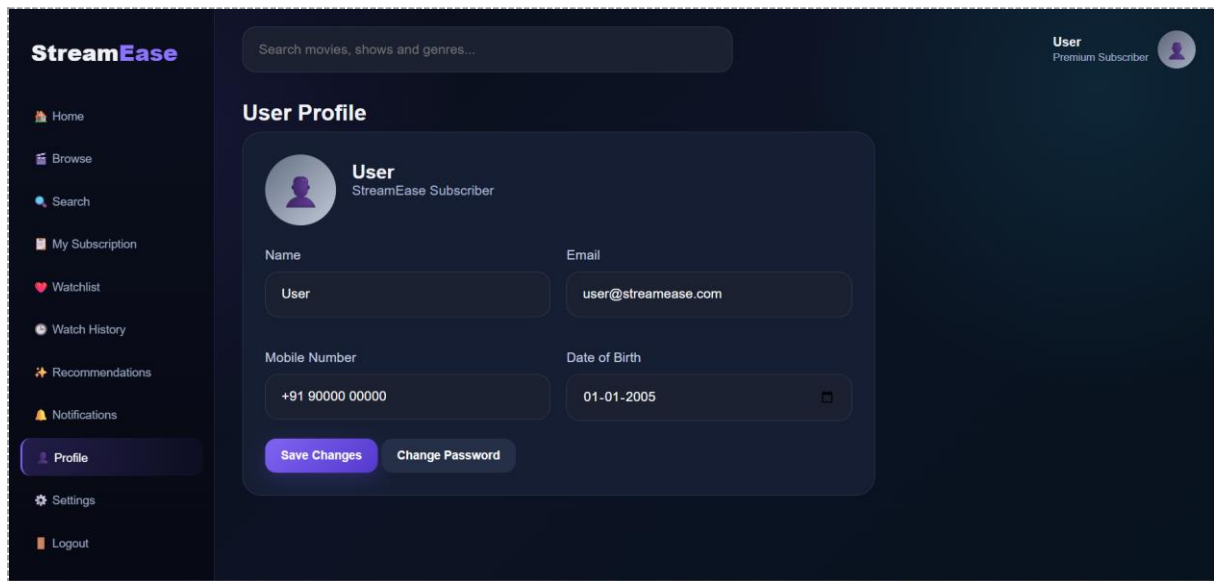
#### 4. Subscription page



#### 5. Watchlist and Viewing history



## 6. Profile and Preferences



The image shows the 'User Profile' page in the StreamEase application. The left sidebar contains navigation links: Home, Browse, Search, My Subscription, Watchlist, Watch History, Recommendations, Notifications, Profile (selected), Settings, and Logout. The main content area has a search bar at the top. Below it, the 'User Profile' section displays a user card for 'User', a StreamEase Subscriber. The card contains input fields for Name (filled with 'User'), Email (filled with 'user@streamease.com'), Mobile Number (filled with '+91 90000 00000'), and Date of Birth (filled with '01-01-2005'). At the bottom of the card are two buttons: 'Save Changes' and 'Change Password'.

**StreamEase** Search movies, shows and genres... User Premium Subscriber

### User Profile

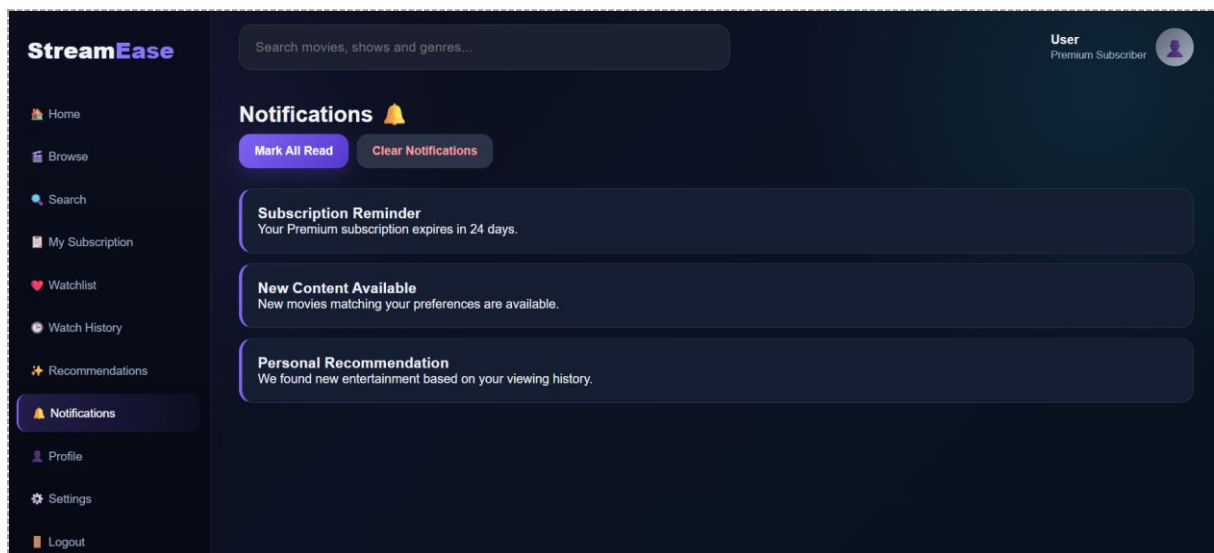
User StreamEase Subscriber

Name: User Email: user@streamease.com

Mobile Number: +91 90000 00000 Date of Birth: 01-01-2005

Save Changes Change Password

## 7. Notifications panel



The image shows the 'Notifications' panel in the StreamEase application. The left sidebar is identical to the previous page, with 'Notifications' selected. The main content area has a search bar at the top. Below it, the 'Notifications' section features a bell icon and two buttons: 'Mark All Read' and 'Clear Notifications'. There are three notification cards: 'Subscription Reminder' (Your Premium subscription expires in 24 days.), 'New Content Available' (New movies matching your preferences are available.), and 'Personal Recommendation' (We found new entertainment based on your viewing history.).

**StreamEase** Search movies, shows and genres... User Premium Subscriber

### Notifications

Mark All Read Clear Notifications

**Subscription Reminder**  
Your Premium subscription expires in 24 days.

**New Content Available**  
New movies matching your preferences are available.

**Personal Recommendation**  
We found new entertainment based on your viewing history.

## 8. One interaction, for example adding a title to the watchlist

